# Exercise F3 – Automated Optimization

Leonardo Antero SP. Yoro

IT Elective IV, BSIT

Jose Rizal University

Mandaluyong, Philippines

leonardoantero.yoro@my.jru.edu

***Abstract* — This research delves into the best possible MiniLM model hyperparameter tuning to perform semantic retrieval on a dataset comprising more than 6,000 book titles and descriptions. By employing Multiple Negatives Ranking Loss (MNRL), the model becomes capable of producing embeddings in which relevant book-query pairs are semantically close to each other. In order to find the optimal values of hyperparameters such as training epochs, weight decay, learning rate, batch size, warmup steps, shuffle, and mixed-precision, the authors systematically applied both Grid Search and Random Search methods. Confronted with Mean Reciprocal Rank (MRR) and Top-K accuracy measures, the experiments demonstrate that Grid Search yielded better results than Random Search in terms of both score stability and magnitude. The optimal setting—10 epochs, weight decay 0.01, test size 0.3, learning rate 3.00E-05, batch size 4, and 500 warmup steps—were able to produce an MRR of 0.9928, Top-1 of 0.9878, as well as Top-3 and Top-5 of 1.0000. The generated embeddings serve as an effective way to recommend books with accurate and contextualized content.**

***Keywords— GRID Search, RANDOM Search, Semantic Retrieval, Multiple Negatives Ranking Loss, Book Recommendation***

## I. Introduction

Natural Language Processing (NLP) has emerged as one of the most powerful portions of artificial intelligence, allowing systems to better understand, interpret, and create human language to the degree of sophistication typically attributed only to people. Pre-trained language models, such as BERT and MiniLM, have triggered a new acceleration for NLP research and applications: pre-trained models are ones that have learned general representations of language from monumental corpus - which means they have learned semantic, syntactic, and contextual relationships in text, using the pre-trained models enables reuse across tasks [1].

Fine-tuning pre-trained models is an essential step when re-purposing them for downstream tasks. Instead of developing a model from scratch, which is expensive in computation and in need of data, fine-tuning uses task-specific labeled data to reorient the model's representations so they appropriate the embedding space to the goals for which they are intended. In retrieval or ranking-based tasks, the goal is not to predict a continuous score, rather the goal of structuring the embedding space is to have relevant document–query pairs mapped close together, while irrelevant document–query pairs remain far apart [1].

The current research work takes a step further in the direction of fine-tuning by applying automated hyperparameter optimization to the MiniLM model with a dataset of book titles and descriptions [3]. The application of Multiple Negatives Ranking Loss (MNRL) facilitates the strengthening of the similarity between positive pairs and the weakening of the similarity with in-batch negatives [2]. The systematic examination of the hyperparameter space through Grid Search and Random Search involves learning rate, batch size, weight decay, number of training epochs, and warmup steps. The evaluation of the improvements in embedding quality and retrieval performance is done through standard information retrieval metrics, especially Mean Reciprocal Rank (MRR) and Top-K recall [1], [2].

## II. Methodology

### *a) Dataset Description*

For this study, the researcher used the publicly available "7k Books – with Metadata" dataset from Kaggle [3]. The source dataset originally had 7,068 books. The dataset provides ample bibliographic and textual information to support semantic recommendations.

In particular, the relevant fields are isbn13, title, subtitle, authors, categories, thumbnail, description, published_year, average_rating, num_pages, and ratings_count. The relevant textual fields are the title and description for generating embeddings. The categorical and numerical fields provide additional context or can also be used to filter.

The variation in book categories, published years, and authorship guarantees diversity in the literary content. This variation allows the model to learn strong representations for semantic similarity to better recommend books aligned with user queries based on content and theme.

A dataset based on literature and metadata is also beneficial for downstream tasks, such as embedding-based similarity. This is because the model can encode

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

contextual and embedded semantic information from the text. This information is essential in constructing a recommendation system that ultimately finds meaningful matches between user queries and the description of books.

### *b) Data Preprocessing*

```
# --- Part 1: Tools and Dataset ---
import pandas as pd
import re

# Load raw dataset
df = pd.read_csv("books.csv")

print("✅ Original Dataset Loaded")
print("Total columns:", len(df.columns))
print(df.columns.tolist())

✅ Original Dataset Loaded
Total columns: 12
['isbn13', 'isbn10', 'title', 'subtitle', 'authors', 'categories', 'thumbnail', 'description', 'published_year', 'average_rating', 'num_pages', 'ratings_count']
```

Fig. 1. Dataset Columns Output

The dataset preprocessing was performed in steps that occurred sequentially to allow processes to be consistent, of high quality, and so that they would not hinder the downstream embedding and regression tasks. The raw dataset was first imported using the pandas library, which provided a means to manipulate and explore it, which would facilitate inspection of the dataset. Upon exploring the dataset, it confirmed there were 22 columns, and understood the name for each column, which illuminated the nature of the metadata and text features that were available.

```
# --- Part 2: Keep Only Relevant Columns ---
columns_to_keep = [
    'isbn13', 'title', 'authors', 'categories',
    'description', 'published_year'
]

# ✅ Keep only these columns (drop everything else)
df = df[[col for col in columns_to_keep if col in df.columns]]
print(f"\n📦 Columns after filtering: {list(df.columns)}")

📦 Columns after filtering: ['isbn13', 'title', 'authors', 'categories', 'description', 'published_year']
```

Fig. 2. Dataset Columns after Filtering

Next, the columns related to semantic modeling were retained. `isbn13`, `title`, `authors`, `categories`, `description`, and `published_year` were the important attributes, and columns that contained redundant or non-text information were removed. The attributes listed above were retained because they represent the important attributes to build a useful semantic representation of books and also minimize the dimensionality of the provided datasets to minimize, among other things, processing time and computational efficiency.

```
# --- Part 3: Handle Missing Values ---
df = df.dropna(subset=['title', 'description'])
df['categories'] = df['categories'].fillna('Uncategorized')
df['authors'] = df['authors'].fillna('Unknown')
```

Fig. 3. Handling Missing Values

Handling missing values was important to preserve data integrity. For records missing the essential text fields, such as title or description, the record was discarded, because both fields would be necessary for creating embeddings. For categories that were non-critical, information was saved by inserting the word "unknown" for missing author text and "uncategorized" in place of missing category labels. This strategy avoided empty cells but also preserved the total number of authors and categories that originally existed in the dataset.

```
# --- Part 4: Clean Text Columns ---
def clean_text(text):
    if not isinstance(text, str):
        return ""
    text = text.lower()  # ✅ normalize casing
    text = re.sub(r'<[^>]+>', ' ', text)  # remove HTML tags
    text = re.sub(r'[^a-z0-9\s.,:;!?()-]', ' ', text)  # remove special chars but keep punctuation
    text = re.sub(r'\s+', ' ', text).strip()  # remove extra spaces
    return text

for col in ['title', 'authors', 'categories', 'description']:
    df[col] = df[col].apply(clean_text)
```

Fig. 4. Special Characters Cleaning

The text in all string columns underwent a cleaning process to help standardize the input for the model. The text was first altered to lower case form to standardize for differing casing, and HTML tags were removed to strip unnecessary formatting artifacts. Special characters that could affect tokenization were converted to whitespace, while punctuation commonly used in natural language, such as periods and commas, was retained. Consecutive whitespace characters were collapsed into a single whitespace character to facilitate uniform tokenization when training the model. This cleaning process was also applied to the title, authors, categories, and description.

```
# --- Part 5: Combine Title + Description for Embedding ---
df['text_for_embedding'] = (df['title'] + " " + df['description']).apply(lambda x: x.lower())
```

Fig. 5. Data Combination for Embeddings preparation

To prepare the data for generating embeddings, a new text field called text_for_embedding was created by concatenating the book title and description. This combined representation allows the model to learn from both shorter title information and longer descriptive information and improve its learning for contextual semantic similarity of different books .

```
# --- Part 6: Final Cleaning ---
df = df.drop_duplicates(subset=['isbn13', 'title'])
df['published_year'] = pd.to_numeric(df['published_year'], errors='coerce').fillna(0).astype(int)
```

Fig. 6. Duplicate Values Removal and Data Type Cleaning

Cleaning was also performed by collapsing any duplicate entries based on the duplicated combination of ISBN-13 and title to ensure that the model is not influenced by additional examples of the same book. Other column entries, such as published_year, which was converted to a numeric type to clean and ensure that any invalid or missing entry was also converted to zero across all numeric columns, so all analyses were applied consistently to numeric types.

```python
# --- Part 7: Save Cleaned Data ---
output_path = "cleaned_books.csv"
df.to_csv(output_path, index=False, encoding='utf-8')

print(f"\n💾 Cleaned dataset saved as: {output_path}")
print("✅ Preprocessing Complete!")
print(df.sample(5))
print("\nTotal Books:", len(df))
```

```
💾 Cleaned dataset saved as: cleaned_books.csv
✅ Preprocessing Complete!
            isbn13                              title  \
1200  9780192835024                  the divine comedy
2111  9780375724503                       ghostwritten
2094  9780375707216               i married a communist
1414  9780307264657                          terrorist
1313  9780231054751  genetics and the origin of species

                                            authors categories  \
1200  dante alighieri;charles hubert sisson;david h....     poetry
2111                                   david mitchell    fiction
2094                                      philip roth    fiction
1414                                      john updike    fiction
1313                               theodosius dobzhansky    science

                                         description  published_year
1200  a new blank verse translation of dante s epic,...            1998
2111  the lives of nine total strangers from nine di...            2001
2094  ira ringold, a ditchdigger from newark, rises ...            1999
1414  eighteen-year-old ahmad, the son of an irish-a...            2006
1313  featuring an introduction by stephen jay gould...            1982

                                   text_for_embedding
1200  the divine comedy a new blank verse translatio...
2111  ghostwritten the lives of nine total strangers...
2094  i married a communist ira ringold, a ditchdigg...
1414  terrorist eighteen-year-old ahmad, the son of ...
1313  genetics and the origin of species featuring a...

Total Books: 6548
```

Fig. 7. Post-Preprocessing Dataset Contents

The cleaned, processed dataset is now saved as cleaned_books.csv. This dataset contains high-quality, structured, and precise information, designed for downstream NLP tasks, such as embedding generation, similarity checks, and regression-based fine-tuning. A sample and check of the dataset confirmed there were no preprocessing errors. The total number of books contained in the dataset was also presented to help indicate the overall final size of the dataset.

### c) *Grid Search and Random Search Code*

```python
# -------------------------
# 1. SETUP AND DATA PREPARATION
# -------------------------
import os
os.environ["WANDB_MODE"] = "disabled"

!pip install -q sentence-transformers datasets
import torch
from torch.utils.data import DataLoader
from sentence_transformers import SentenceTransformer, losses, InputExample
import pandas as pd
from sklearn.model_selection import train_test_split, ParameterGrid, ParameterSampler
from sklearn.metrics.pairwise import cosine_similarity
from tqdm import tqdm
import random

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

# Load dataset
csv_path = "/content/cleaned_books.csv"
df = pd.read_csv(csv_path)
df = df[['title', 'text_for_embedding']].dropna()
df['title'] = df['title'].astype(str)
df['text_for_embedding'] = df['text_for_embedding'].astype(str)
```

Fig. 8. Setup and Data Preparation

The workflow commences with the setup and import of necessary libraries. These include sentence-transformers for using the MiniLM model, torch for GPU-accelerated computation, and pandas for dataset handling. The computing environment is set up to automatically detect if CUDA is available, thus enabling an efficient GPU utilization during training. To speed up the execution and keep it reproducible, logging with Weights & Biases is turned off. The dataset, which is a local CSV file after being preprocessed, has the information of the books in the form of their titles and descriptions. Only the relevant text columns are retained, and all entries are converted into string format to ensure compatibility with the tokenizer and embedding model. By doing so, the data is made ready which also helps in avoiding any type-related errors during the stages of fine-tuning and embedding generation.

```python
# -------------------------
# 2. DEFINE PARAMETER GRIDS
# -------------------------
grid_param = {
    'num_train_epochs': [8, 10],
    'learning_rate': [3e-5],
    'weight_decay': [0.01, 0.3],
    'per_device_train_batch_size': [4, 8],
    'warmup_steps': [500],
    'fp16': [True],
    'shuffle': [True],
    'test_size': [0.3]
}

# -------------------------
# RANDOM SEARCH (unchanged)
# -------------------------
random_param = {
    'num_train_epochs': [2, 4, 6, 8, 10, 12],
    'learning_rate': [1e-5, 2e-5, 3e-5, 4e-5, 5e-5],
    'weight_decay': [0, 0.01, 0.05, 0.1, 0.3, 0.4],
    'per_device_train_batch_size': [4, 8, 12, 16, 24, 32],
    'warmup_steps': [100, 200, 300, 500, 800, 1000],
    'fp16': [True, False],
    'shuffle': [True, False],
    'test_size': [0.1, 0.2, 0.3]
}
n_random_samples = 10  # number of random runs
```

Fig. 9. Parameter Definition for both Techniques

Grid search and random search are the two hyperparameter optimization strategies that have been utilized. The grid search sets up a structured set of candidate values for each hyperparameter, thus focusing on the controlled exploration of the configurations that have been identified as potentially high-performing. On the other hand, the random search picks the configurations from a broader parameter range, thus allowing the discovery of less obvious but possibly better settings. The hyperparameters are the number of training epochs, learning rate, batch size, warmup steps, weight decay, FP16 mixed-precision training, data shuffling, and test split ratio. By defining both search spaces, it is possible to systematically and comparatively evaluate the sensitivity of the model to parameter variation and the resulting effect on convergence and retrieval accuracy.

```python
# -------------------------
# 3. MODEL TRAINING FUNCTION
# -------------------------
def train_and_evaluate(params, df):
    # Split dataset
    train_df, val_df = train_test_split(df, test_size=params['test_size'], random_state=42, shuffle=params['shuffle'])

    # Load model
    model_name = "sentence-transformers/all-MiniLM-L6-v2"
    model = SentenceTransformer(model_name)
    model = model.to(device)

    # Prepare InputExamples
    train_examples = [InputExample(texts=[row['text_for_embedding'], row['title']])
                      for _, row in train_df.iterrows()]
    train_loader = DataLoader(train_examples, shuffle=True, batch_size=params['per_device_train_batch_size'])
    mnrl_loss = losses.MultipleNegativesRankingLoss(model=model)

    # Train
    model.fit(
        train_objectives=[(train_loader, mnrl_loss)],
        epochs=params['num_train_epochs'],
        optimizer_params={'lr': params['learning_rate'], 'weight_decay': params['weight_decay']},
        show_progress_bar=True
    )

    # Evaluate
    metrics = evaluate_mnrl(model, val_df, train_df)
    return model, metrics
```

Fig. 10. Model Training Code

The train_and_evaluate() function is the main controller for the model training process given a single hyperparameter configuration. First of all, the dataset is split into the train and validation sets according to the test_size and shuffle parameters. Every (title, description) pair is wrapped into an InputExample instance that is a common format for contrastive learning under the Multiple Negatives Ranking Loss (MNRL) objective. The batching of data is achieved via a DataLoader to make memory and gradient calculation more efficient. The MiniLM model is fine-tuned through the specified number of epochs with the given learning rate and weight decay parameters. After that, the model is tested on the validation set to get the main retrieval metrics that give a quantitative insight into the quality of the embeddings and the ranking performance.

```python
# -------------------------
# 4. EVALUATION FUNCTION
# -------------------------
def evaluate_mnrl(model, df_query, df_corpus, top_k=(1,3,5)):
    model.eval()
    queries = df_query['text_for_embedding'].tolist()
    query_titles = df_query['title'].tolist()
    corpus_texts = df_corpus['text_for_embedding'].tolist()
    corpus_titles = df_corpus['title'].tolist()
    corpus_emb = model.encode(corpus_texts, convert_to_tensor=True, show_progress_bar=True)

    top1 = top3 = top5 = mrr_total = valid_count = 0
    for q_text, q_title in tqdm(zip(queries, query_titles), total=len(queries)):
        q_emb = model.encode(q_text, convert_to_tensor=True)
        sims = cosine_similarity(q_emb.cpu().numpy().reshape(1,-1), corpus_emb.cpu().numpy())[0]
        ranked_idx = sims.argsort()[::-1]
        ranked_titles = [corpus_titles[i] for i in ranked_idx]
        if q_title not in ranked_titles:
            continue
        rank = ranked_titles.index(q_title) + 1
        mrr_total += 1.0 / rank
        valid_count += 1
        if q_title in ranked_titles[:1]: top1 += 1
        if q_title in ranked_titles[:3]: top3 += 1
        if q_title in ranked_titles[:5]: top5 += 1

    if valid_count == 0: return {"MRR": 0, "Top1":0, "Top3":0, "Top5":0}
    return {"MRR": mrr_total/valid_count, "Top1": top1/valid_count, "Top3": top3/valid_count, "Top5": top5/valid_count}
```

Fig. 11. Model Evaluation Code

The evaluate_mnrl() function tests a model's semantic retrieval ability. It takes corpus and query texts, both of which are then transformed into embeddings with a fine-tuned model. After that, pairwise cosine similarities between the embeddings are calculated. The similarity scores thus obtained are used to sort the corpus for each query, thereby allowing the calculation of Mean Reciprocal Rank (MRR) and Top-K accuracy (Top-1, Top-3, and Top-5) metrics. These metrics measure the model's capability to bring the correct document at the top of the most highly-ranked results. Such an evaluation is essentially a confirmation of the model's proficiency in identifying semantic connections in two given natural language expressions, which is a prerequisite for content-based recommendation to be dependable.

```python
# -------------------------
# 5. RUN GRID SEARCH
# -------------------------
print("=== GRID SEARCH ===")
for params in ParameterGrid(grid_param):
    print("\nRunning parameters:", params)
    model, metrics = train_and_evaluate(params, df)
    print("Metrics:", metrics)
```

Fig. 12. Grid Search Hyperparameter Runs

Grid search goes through each and every possible combination of hyperparameters one by one in a very systematic way. For each configuration, a separate training and evaluation are done followed by logging the performance metrics for later comparison. It is basically an exhaustive and reproducible way to assess the interactions of parameters which makes it easy to find the configuration that gives the highest MRR and Top-K scores. By limiting the variables involved in the experiment, grid search really helps in understanding the impact of specific parameters like batch size, number of epochs, or weight decay on the performance and generalization of the model.

```python
# -------------------------
# 6. RUN RANDOM SEARCH
# -------------------------
print("=== RANDOM SEARCH ===")
random_combos = list(ParameterSampler(random_param, n_iter=n_random_samples, random_state=42))
for params in random_combos:
    print("\nRunning parameters:", params)
    model, metrics = train_and_evaluate(params, df)
    print("Metrics:", metrics)
```

Fig. 13. Random Search Hyperparameter Runs

Random search works with grid search by picking parameter combinations at random from a larger hyperparameter space. Each chosen configuration is then trained and tested using the same experimental protocol as grid search. Such a stochastic method makes it possible to find good configurations quicker, especially when the parameter space is huge and a full evaluation is very costly in terms of computation. By varying the search, random search uncovers a wider performance landscape of the model, which shows the stability of MiniLM under different training conditions and saves computation time during hyperparameter tuning.

### *d) MiniLM Fine-Tune Model*

This project enhances a MiniLM sentence embedding model by refining it to establish the semantic similarity of book titles and their text description. The goal of the project is to ultimately obtain vector representations that will allow for relevant books to be retrieved based on contextual similarity to work as a straight-forward content based recommendation system. The model is fine-tuned using contrastive learning, by pulling positive pairs closer to one another in the embedding space and using unrelated samples as natural negatives against the relevant book pairings.

```python
# -------------------------
# 1. SETUP AND DATA PREPARATION
# -------------------------
import os
os.environ["WANDB_MODE"] = "disabled"  # Disable wandb

!pip install -q sentence-transformers datasets
import torch
from torch.utils.data import DataLoader
from sentence_transformers import SentenceTransformer, losses, InputExample
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics.pairwise import cosine_similarity
from tqdm import tqdm

# Check GPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

# Load dataset
csv_path = "/content/cleaned_books.csv"
df = pd.read_csv(csv_path)
print(f"Dataset loaded: {df.shape[0]} rows")

# Keep only necessary columns and convert to string
df = df[['title', 'text_for_embedding']].dropna()
df['title'] = df['title'].astype(str)
df['text_for_embedding'] = df['text_for_embedding'].astype(str)
```

Fig. 14. Importing of Dependencies and Installation

The first step in the process, is to install and import all libraries that are required, which are used to load and train the model, torch to use GPU-accelerated computation, and pandas for working with the datasets. A device check is also used to test to see if there is CUDA available and if there is, the training process will utilize the GPU hardware to efficiently improve the length and speed of computing. Lastly, researchers make sure to turn off logging using Weights & Biases , to allow us to train without making logs of the results.

The dataset is read from a pre-processed csv file that has the book titles in addition to the text field. The code only keeps the columns needed and all the entries have been changed into string format as indicated by tokenizer. This step allows the model to have a clean text input for embedding generations, and prevents potential data-type errors during training.

```python
# -------------------------
# 2. TRAIN/VALIDATION SPLIT
# -------------------------
train_df, val_df = train_test_split(df, test_size=0.1, random_state=42, shuffle=True)
print(f"Train size: {len(train_df)}, Validation size: {len(val_df)}")

# -------------------------
# 3. MODEL DEFINITION
# -------------------------
MODEL_NAME = "sentence-transformers/all-MiniLM-L6-v2"
model = SentenceTransformer(MODEL_NAME)
model = model.to(device)
print(f"Model loaded: {MODEL_NAME}")
```

Fig. 15. Train/Test Split and Model Definition

The dataset is split into training and validation subsets at a ratio of 90:10 so that the model is assessed on unseen data during training. A fixed random seed is used for reproducibility, and the shuffle parameter ensures that samples are randomized instead of being sorted by the order they were introduced.

Next, a MiniLM sentence-embedding model pre-trained (all-MiniLM-L6-v2) is loaded in the framework SentenceTransformers. This efficient transformer architecture is well-balanced between performance and computational efficiency, making it suitable for large-scale embedding generation without being unrealistic with limited GPU resources. After initialization, the model is moved to a device, either a CPU or a different device with CUDA, to prepare for optimization and fine-tuning.

```python
# -------------------------
# 4. HYPERPARAMETERS
# -------------------------
num_train_epochs = 3
per_device_train_batch_size = 16
warmup_steps = 1000
weight_decay = 0.01
logging_steps = 100
fp16 = torch.cuda.is_available()
learning_rate = 3e-5
```

Fig. 16. Baseline Model Hyperparameters

In order for the embedding space to converge while minimizing overfitting, the model is trained for 3 epochs, allowing multiple passes through the dataset for the embeddings. A batch size of 16 balances GPU memory restrictions with enough negative examples in a batch for the Multiple Negatives Ranking Loss. The training schedule also includes 1000 warmup steps where training begins with a low learning rate and increases during the warmup, in order to avoid destabilizing the pre-trained MiniLM weights. In order to aid generalization and avoid overfitting during training, a weight decay of 0.01 is used as L2 regularization. Training update logs are comparatively less frequent, every 100 logging steps, which gives some awareness of training without slowing down the overall training process. FP16 mixed precision is automatically enabled if a GPU is present to save memory and maximize speed while training. Lastly, the learning rate is set to 3e-5 which has been effective for transformer fine-tuning in the past allowing fine control of the adaptation of the model's parameters.

```python
# -------------------------
# 5. PREPARE INPUTEXAMPLES AND DATALOADER
# -------------------------
# Multiple Negatives Ranking Loss setup
train_examples = [InputExample(texts=[row['text_for_embedding'], row['title']])
                  for _, row in train_df.iterrows()]
val_examples = [InputExample(texts=[row['text_for_embedding'], row['title']])
                for _, row in val_df.iterrows()]

train_loader = DataLoader(train_examples, shuffle=True, batch_size=per_device_train_batch_size)

# Loss function
mnrl_loss = losses.MultipleNegativesRankingLoss(model=model)
```

Fig. 17. DataLoader preparation

The dataset is transformed into training samples in pairs where each entry has the book description and its corresponding title, forming a positive semantic pair. These pairs are organized as InputExample objects for

both the training and validation carets, ensuring that each example sample represents a valid text-to-title relationship for the model. Next, a DataLoader is created to batch and shuffle the data during training; every batch naturally provides its own negative examples because every other title in the batch is treated as a non-matching example. The model is trained with the Multiple Negatives Ranking Loss, which is a contrastive objective that increases the similarity between true description–title pairs while decreasing similarity with unrelated pairs. The model is thereby trained to retrieve embeddings for its titles that will be useful for retrieval purposes, without requiring any negative examples specified manually; they are all automatically generated in each training batch.

```python
# -------------------------
# 6. FINE-TUNING
# -------------------------
model.fit(
    train_objectives=[(train_loader, mnrl_loss)],
    epochs=num_train_epochs,
    optimizer_params={'lr': learning_rate, 'weight_decay': weight_decay},
    show_progress_bar=True,
    callback=None
)

save_path = "/content/mnrl_finetuned_books"  # folder name where model will be stored
model.save(save_path)
print(f"✅ Model saved to: {save_path}")
```

Fig. 18. Fine-tuning Phase

The fine-tuning phase trains the MiniLM model using the DataLoader that was prepared previously and the Multiple Negatives Ranking Loss as the training objective. The fit function executes the complete training loop applying the predetermined number of epochs, learning rate, and weight decay through the optimizer settings. During training, the model continuously updates its embedding space such that matching description–title pairs pull together while unassociated titles push away from each other. A progress bar is enabled to show training progress, but no callback functions are defined, allowing the training process to focus on ranking as its main objective. After training is finished, and model weights and tokenizer configuration have been saved to the respective directory, the fine-tuned model can be loaded later for inference or to run evaluation.

```python
# -------------------------
# 7. EVALUATION METRICS
# -------------------------
def evaluate_mnrl(model, df_query, df_corpus, top_k=(1,3,5)):
    model.eval()
    queries = df_query['text_for_embedding'].tolist()
    query_titles = df_query['title'].tolist()

    corpus_texts = df_corpus['text_for_embedding'].tolist()
    corpus_titles = df_corpus['title'].tolist()

    corpus_emb = model.encode(corpus_texts, convert_to_tensor=True, show_progress_bar=True)

    top1 = top3 = top5 = mrr_total = valid_count = 0
    for q_text, q_title in tqdm(zip(queries, query_titles), total=len(queries)):
        q_emb = model.encode(q_text, convert_to_tensor=True)
        sims = cosine_similarity(q_emb.cpu().numpy().reshape(1,-1), corpus_emb.cpu().numpy())[0]
        ranked_idx = sims.argsort()[::-1]
        ranked_titles = [corpus_titles[i] for i in ranked_idx]

        # Skip if correct title not in corpus
        if q_title not in ranked_titles:
            continue

        rank = ranked_titles.index(q_title) + 1
        mrr_total += 1.0 / rank
        valid_count += 1

        if q_title in ranked_titles[:1]:
            top1 += 1
        if q_title in ranked_titles[:3]:
            top3 += 1
        if q_title in ranked_titles[:5]:
            top5 += 1

    if valid_count == 0:
        return {"MRR": 0, "Top1": 0, "Top3": 0, "Top5": 0}

    return {
        "MRR": mrr_total / valid_count,
        "Top1": top1 / valid_count,
        "Top3": top3 / valid_count,
        "Top5": top5 / valid_count
    }

metrics = evaluate_mnrl(model, val_df, train_df)  # Use validation queries against training corpus
print("Evaluation Metrics:")
for k, v in metrics.items():
    print(f"{k}: {v:.4f}")
```

Fig. 19. Evaluation Results

The evaluation stage gauges the fine-tuned model's efficacy for retrieving the appropriate title given the description as a query. The queries come from the validation set and the training set serves as the corpus to be searched, which imitates a real recommendation scenario. The model first encodes each title in the corpus into embeddings, then encodes each query and calculates a cosine similarity with every book embedding to produce a ranked list of candidate titles. Ranking performance is quantified with four retrieval metrics: Mean Reciprocal Rank (MRR), which takes the mean of the inverse rank of the correct title for each query, and provides top-1, top-3, and top-5 accuracy, which represent the percentage of times the correct title occurs in the top-1, top-3, and top-5 results, respectively. These metrics together measure total ranking performance and practical usefulness in terms of finding the correct book title in the top recommendation suggestions overall.

### *e) Inference Process and Embeddings*

```python
import torch
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
import pandas as pd

# -------------------------------------------------
# 1. Load fine-tuned model
# -------------------------------------------------
model_path = "/content/mnrl_finetuned_books"  # same folder as saved above
model = SentenceTransformer(model_path)
model = model.to("cuda" if torch.cuda.is_available() else "cpu")
print("✅ Fine-tuned model loaded!")
```

Fig. 20. Loading of Fine-Tuned Model

The inference phase starts by loading the fine-tuned MiniLM model from the earlier saved folder,

restoring both the weights of the transformer and its embedding configuration which were used during training. The model is automatically set to run on available hardware, favoring the GPU when available, to expedite the process of computing an embedding. Once loaded the model can now convert any new book description into vector representations, allowing for similarity search in the existing corpus. This is the beginning of the model providing a full recommendation pipeline, with the embedding space being used for real-world retrieval, rather than for more optimization.

```
# ---------------------------------------------------
# 2. Load your reference corpus (titles + embeddings)
# ---------------------------------------------------
corpus_df = pd.read_csv("/content/cleaned_books.csv")  # your cleaned dataset
corpus_texts = corpus_df["text_for_embedding"].tolist()
corpus_titles = corpus_df["title"].tolist()

# Precompute embeddings once (for speed)
print("Encoding corpus embeddings (this may take a few minutes)...")
corpus_embeddings = model.encode(corpus_texts, convert_to_tensor=True, show_progress_bar=True)
```

Fig. 21. Embeddings Computation

The searchable database for recommendations is loaded and saved as a reference corpus consisting of all available titles in the corpus, and the title's corresponding description text. The text fields are extracted into lists for encoding by the model with the title preserved for recall once ranking is complete. The system computes the embedding for each book in the corpus only once with the fine-tuned model to streamline inference. It caches either in memory or on storage so that models do not need to encode again during user queries, and only a cosine similarity lookup is needed for retrieval. This means the system can operate in real time for queries, regardless of reference corpus size after the initial embeddings are generated and stored, which is necessary in practice for recommendation engines.

```
# ---------------------------------------------------
# 3. Interactive Inference: User Input Query
# ---------------------------------------------------
while True:
    user_query = input("\nEnter your search query (or type 'exit' to quit): ")
    if user_query.lower() == "exit":
        break

    # Encode the user query
    query_emb = model.encode(user_query, convert_to_tensor=True)

    # Compute cosine similarity between query and corpus
    sims = cosine_similarity(query_emb.cpu().numpy().reshape(1, -1),
                             corpus_embeddings.cpu().numpy())[0]

    # Sort and get top-k results
    top_k = 5
    top_indices = sims.argsort()[-top_k:][::-1]

    print("\n🔍 Top Relevant Books:")
    for i, idx in enumerate(top_indices, start=1):
        print(f"{i}. {corpus_titles[idx]}  (Score: {sims[idx]:.4f})")
```

Fig. 22. User Input Query

During the last stage of the pipeline, the user can real-time retrieve with a text query (a book description or topic of interest), which is encoded into an embedding, and its cosine similarity calculated from the query vector with all of the pre-computed corpus embeddings. The scores are then ranked in descending order and the recommendations to the user will reflect the highest scoring and normally relevant scores. In this case, the interface provides the Top-5, along with their similarity scores, to demonstrate that the model has performed content-based searching without manual tagging, genre filtering or pre-defined categories. This concludes the full retrieval pipeline by illustrating how the implementation of contrastive fine-tuning can translate directly to a user experience for recommendation

.

## III. Results

The fine-tuning the model experiments consisted of eighteen individual runs of the MiniLM model, split into two search strategies: Grid Search and Random Search. Each run experimented with a different set of hyperparameters, mainly the number of epochs for training, weight decay, and test size, while the rest of the parameters were kept unchanged to control the effects of each variable. The performance measures were Mean Reciprocal Rank (MRR) and Top-K accuracies at one, three, and five, which indicate how well the fine-tuned model can find the correct book titles using semantic similarity. In general, the model was able to achieve very high retrieval precision most of the time, with several instances of almost perfect results, thus, it can be said that MiniLM is quite robust in generating semantically rich embeddings for recommendation tasks.

### *a) Grid Search*

The Grid Search experiments systematically explored various controlled parameter combinations, mainly changing the number of epochs, batch size, and weight decay. The results indicated that models trained for eight to ten epochs generally achieved high retrieval accuracy. The optimal performance was noted in the setting where there were eight epochs, a batch size of four, a weight decay of 0.01, and a test size of 0.3. The model was able to reach a Mean Reciprocal Rank of 0.9928 along with a Top-1 accuracy of 0.9878. Moreover, Top-3 and Top-5 accuracies were able to achieve perfect scores of 1.0. In a sense, these results show that moderate but extended training rounds lead to stable convergence as well as high-quality semantic representations.

The experiments with weight decay values between 0.01 and 0.3 showed that there were very slight differences in results, which led to the conclusion that MiniLM can still perform stably under mild regularization. The configurations which were using larger batch sizes of eight were slightly lowering MRR, thus small batches must be giving more frequent gradient updates and making it possible to get finer adjustments during training. Grid Search was able to show that longer training and smaller batch sizes resulted in the most accurate and consistent retrieval performance. This method aligns with standard practices of hyperparameter tuning for structured search spaces, which are documented to yield reliable performance comparisons [4].

| Run ID | Search Type | Date | Time | Split | learning_rate | num_train_epochs | per_device_train_batch_size | shuffle | test_size | warmup_steps | weight_decay | runtime | MRR | Top 1 | Top 3 | Top 5 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | GRID | 5/5/2025 | 12:38 P.M. | TRUE | 3.00E-05 | 8 | 4 | TRUE | 0.3 | 500 | 0.01 | 2hrs, 5 mins | 0.9561 | 0.9475 | 1 | 1 |
| 2 | GRID | 5/5/2025 | 12:38 P.M. | TRUE | 3.00E-05 | 8 | 4 | TRUE | 0.3 | 500 | 0.3 | 2hrs, 5 mins | 0.9526 | 0.9476 | 1 | 1 |
| 3 | GRID | 5/5/2025 | 12:38 P.M. | TRUE | 3.00E-05 | 8 | 8 | TRUE | 0.3 | 500 | 0.01 | 2hrs, 5 mins | 0.9576 | 0.9451 | 0.9929 | 0.9929 |
| 4 | GRID | 5/5/2025 | 12:38 P.M. | TRUE | 3.00E-05 | 8 | 8 | TRUE | 0.3 | 500 | 0.3 | 2hrs, 5 mins | 0.9676 | 0.9451 | 0.9929 | 0.9929 |
| 5 | GRID | 5/5/2025 | 12:38 P.M. | TRUE | 3.00E-05 | 10 | 4 | TRUE | 0.3 | 500 | 0.01 | 2hrs, 5 mins | 0.9667 | 0.9526 | 0.9929 | 0.9929 |
| 6 | GRID | 5/5/2025 | 12:38 P.M. | TRUE | 3.00E-05 | 10 | 4 | TRUE | 0.3 | 500 | 0.3 | 2hrs, 5 mins | 0.9667 | 0.9576 | 0.9929 | 0.9929 |
| 7 | GRID | 5/5/2025 | 12:38 P.M. | TRUE | 3.00E-05 | 10 | 8 | TRUE | 0.3 | 500 | 0.01 | 2hrs, 5 mins | 0.9781 | 0.9634 | 0.9929 | 0.9929 |
| 8 | GRID | 5/5/2025 | 12:38 P.M. | TRUE | 3.00E-05 | 10 | 8 | TRUE | 0.3 | 500 | 0.3 | 2hrs, 5 mins | 0.9781 | 0.9634 | 0.9929 | 0.9929 |

Fig. 23. Grid Search Runs Excel Logs

*b) Random Search*

The Random Search experiments examined a wider and more varied range of hyperparameter combinations than the Grid Search experiments. The Random Search experiments included changing of learning rate, warmup steps, batch size, mixed-precision flag, and test split ratio. Random search differs from grid search in that it can consider less logical parameter combinations that have not been considered before and can also more quickly find high-performing configurations when the search space is large, as described in tuning methodologies [4].

In this study, a number of random configurations resulted in retrieval performance at the level of or even higher than the benchmark, with several runs producing MRR scores exceeding 0.95 and likewise high Top-K accuracies. The best setup under Random Search was able to achieve an MRR of 0.978 and a Top-1 accuracy of 0.9561 by using ten epochs, a batch size of four, weight decay of 0.01, and a test size of 0.2.

Intuitively, models that were trained for at least ten epochs and that had a small weight decay and a moderate batch size were able to establish significant semantic relationships between queries and the corresponding titles of books. On the contrary, shorter training durations, especially at two epochs, seemed to suffer from underfitting as the MRR values were below 0.90. This fact points to the necessity of an adequate training time if one wants the model to be able to grasp the semantic relations effectively.

Moreover, lower learning rates allowed smoother model convergence whereas the use of mixed-precision training did not affect the model's accuracy but rather improved runtime efficiency. The results obtained from the Random Search experiment show that the MiniLM model is capable of generalizing well across a range of different hyperparameter settings and also demonstrate the benefit of stochastic search strategies in terms of efficiency when it comes to locating optimal parameter combinations without the need for a full exploration of the search space.

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 9 | RANDOM | 5/5/2025 | 12:39 P.M. | TRUE | 1.00E-05 | 2 | 16 | FALSE | 0.3 | 1000 | 0.05 | 2hrs, 5 mins | 0.7632 | 0.5921 | 0.9539 | 0.9539 |
| 10 | RANDOM | 5/5/2025 | 12:39 P.M. | FALSE | 6.00E-05 | 12 | 8 | FALSE | 0.1 | 1000 | 0.09 | 2hrs, 5 mins | 0.8868 | 0.8026 | 1 | 1 |
| 11 | RANDOM | 5/5/2025 | 12:39 P.M. | FALSE | 8.00E-05 | 2 | 12 | TRUE | 0.1 | 500 | 0.3 | 2hrs, 5 mins | 0.8882 | 0.8187 | 0.9539 | 1 |
| 12 | RANDOM | 5/5/2025 | 12:39 P.M. | TRUE | 1.00E-05 | 10 | 32 | TRUE | 0.1 | 500 | 0.01 | 2hrs, 5 mins | 0.822 | 0.6798 | 1 | 1 |
| 13 | RANDOM | 5/5/2025 | 12:39 P.M. | TRUE | 6.00E-05 | 10 | 24 | TRUE | 0.2 | 200 | 0 | 2hrs, 5 mins | 0.914 | 0.82 | 1 | 1 |
| 14 | RANDOM | 5/5/2025 | 12:39 P.M. | FALSE | 1.00E-05 | 10 | 4 | TRUE | 0.2 | 1000 | 0.01 | 2hrs, 5 mins | 0.8176 | 0.6901 | 1 | 1 |
| 15 | RANDOM | 5/5/2025 | 12:39 P.M. | FALSE | 3.00E-05 | 10 | 12 | FALSE | 0.2 | 1000 | 0.1 | 2hrs, 5 mins | 0.9023 | 0.811 | 0.9921 | 0.9921 |
| 16 | RANDOM | 5/5/2025 | 12:39 P.M. | TRUE | 3.00E-05 | 2 | 12 | TRUE | 0.2 | 500 | 0.1 | 2hrs, 5 mins | 0.9203 | 0.8171 | 0.9592 | 0.9921 |
| 17 | RANDOM | 5/5/2025 | 12:39 P.M. | FALSE | 1.00E-05 | 4 | 24 | TRUE | 0.2 | 200 | 0.09 | 2hrs, 5 mins | 0.9167 | 0.8293 | 0.9585 | 0.9042 |

Fig. 24. Random Search Runs Excel Logs

*c) Query-Based Inference*

```
✅ Fine-tuned model loaded!
Encoding corpus embeddings (this may take a few minutes)...
Batches: 100% |▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰| 205/205 [00:05<00:00, 92.63it/s]

Enter your search query (or type 'exit' to quit): don quixote

🔍 Top Relevant Books:
1. don quixote  (Score: 0.9213)
2. mozart s don giovanni  (Score: 0.3585)
3. little dorrit  (Score: 0.3364)
4. the lorax  (Score: 0.3239)
5. fools die  (Score: 0.2997)

Enter your search query (or type 'exit' to quit): does the library has materials about programming

🔍 Top Relevant Books:
1. the code book  (Score: 0.3628)
2. structure and interpretation of computer programs  (Score: 0.3607)
3. programming ruby  (Score: 0.3555)
4. joel on software  (Score: 0.3463)
5. c programmer s notebook  (Score: 0.3206)

Enter your search query (or type 'exit' to quit): preacher writings

🔍 Top Relevant Books:
1. the gospel according to job  (Score: 0.4108)
2. a commentary upon the gospel according to s. luke  (Score: 0.4084)
3. writings, 1902-1910  (Score: 0.3826)
4. the gospel according to tolkien  (Score: 0.3669)
5. lost prophet  (Score: 0.3562)

Enter your search query (or type 'exit' to quit): academic materials for literacy

🔍 Top Relevant Books:
1. teaching reading comprehension to students with learning difficulties  (Score: 0.3726)
2. active literacy across the curriculum  (Score: 0.3415)
3. how to read literature like a professor  (Score: 0.3352)
4. his dark materials  (Score: 0.3305)
5. scholastic success with 4th grade(workbook)  (Score: 0.3293)

Enter your search query (or type 'exit' to quit): japanese books

🔍 Top Relevant Books:
1. japan  (Score: 0.6560)
2. japan at war  (Score: 0.5777)
3. yakitate!! japan  (Score: 0.4951)
4. book of haikus  (Score: 0.4786)
5. china and japan  (Score: 0.4753)

Enter your search query (or type 'exit' to quit): exit
```

Fig. 25. User Query and Recommendation Testing

First, the MiniLM model after being fine-tuned was put through a test that involved interaction-based semantic retrieval. In such a test, queries by users are encoded and then for corpus vectors that were already computed, the nearest vectors to the query vectors are found via cosine similarity. The result shows that the model is able to find the Don Quixote as the most semantically relevant document when the input query is "don quixote" with a similarity measure of 0.9213. After that, it returned the somewhat related works of literature. For example, Mozart's Don Giovanni and Little Dorrit etc., which shows that the retrieval is very precise for exactly matching the title of the work. The model has also surfaced very relevant documents including The Code Book, Structure and Interpretation of Computer Programs, and Programming Ruby in response to a question like "Does the library have any materials related to programming". This showcases how the model is capable of detecting thematic relationships in the field of programming and computing concepts.

For the vague and broad query like "preacher writings", the system came up with the following works that are semantically relevant and also reflect the model's ability to capture theological and reflective contexts beyond just the keyword overlap - The Gospel According to Job, A Commentary upon the Gospel According to S. Luke, and Writings, 1902–1910. In the same fashion, the model for the education-focused query "academic materials for literacy" has fetched the titles that are pedagogically relevant such as Teaching Reading Comprehension to Students with Learning Difficulties and

Active Literacy Across the Curriculum, which is a testimony of the model's capability in educational content retrieval. Also, the query "Japanese books" has accounted for very relevant results such as Japan, Japan at War, and Yakitate!! Japan with the semantic similarity scores ranging from 0.65 to 0.47 approximately, thus it is effectively exemplified that the model grasps the geographical, cultural, and linguistic contexts. In essence, these retrievals from the fine-tuned MiniLM model are evidence that it is able to generate contextually accurate, semantically coherent, and domain-relevant suggestions not only for narrow user queries but also for broad ones. This, in turn, is a testament to their viability for real-world semantic search and recommender system use cases.

## IV. Discussion

### a) *Hyperparameter Effects*

The hyperparameter set of experiments looked at the effects of various variables such as the learning rate, the number of epochs, batch size, test split ratio, weight decay, warmup steps, shuffle, and mixed-precision training on the retrieval performance of the fine-tuned MiniLM model. The Grid Search results showed that systematically trying out a smaller, controlled set of parameter combinations led to retrieval performance that was consistently high. Models trained for eight to ten epochs, batch sizes of four or eight, test sizes of 0.3, and weight decay values between 0.01 and 0.3 were able to reach MRR values as high as 0.9928 and Top-K accuracies close to one. Grid Search allowed pinpointing of the optimal configurations easily and in a reproducible manner, however, it also implied that all combinations had to be evaluated which caused an increase in the computational time [4].

Random Search, conversely, delved into a wider and more varied hyperparameter space. It included such things as extreme learning rates, large batch sizes, and a wide range of test split and weight decay. Despite the fact that sampling was stochastic, Random Search was still able to find high-performing configurations very quickly. Ten or more epochs of training, a moderate batch size, and a small weight decay were conditions that led to strong results, MRR values above 0.95, and Top-1 accuracies exceeding 0.92 [4]. Shorter training runs of two epochs generally resulted in underfitting, thus the importance of sufficient training duration was established [7].

Grid Search was the method that most of the time led to the top retrieval performance in terms of MRR and Top-K accuracy. The results indicate that a systematic evaluation of all parameter combinations can very reliably find the optimal hyperparameter configurations.

Random Search is a flexible method and can quickly explore a wider range of configurations. However, it sometimes produced less optimal results in this experiment, pointing out that stochastic sampling may not find the best settings when the search space is of moderate size. Thus, Grid Search is the instrument of choice for this dataset as it guarantees reproducibility and makes retrieval performance the best, while Random Search can still be handy for large or complex search spaces where a full evaluation is not feasible [4], [5], [6].

### b) *Evaluation Metrics*

The values for MRR and Top-K classification accuracy provide an insightful measure of the quality of the learned embedding's performance between the retrieval model and the embeddings learned over the book dataset. The MRR value being higher suggests that the relevant title(s) for each query are placed most frequently near the top of the K fold (k = 1, 3, 5). Additionally, Top 1, Top 3, and Top 5 classification accuracies, measure the probability of retrieving the correct book title within the top-k once again. The heatmaps and quantitative logs confirmed that hyperparameters achieved a conscientious balance between the quality of the embeddings and the retrieval model performance while semantically similar book titles would be approximated over the dataset. These metrics demonstrate that the fine-tuned MiniLM model can offer consistent, context-aware recommendations [5], [6], [7].

### c) *Data Visualizations*

| | Run ID | Search Type | Date | Time | lr0 | warmup_ratio | num_train_epochs | per_device_train_batch_size | shuffle | test_size | warmup_steps | weight_decay | runtime | MRR | Top 1 | Top 3 | Top 5 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| BEST SPEED | 1 | GRID | 13/3/2025 | 12:39 PM | TRUE | 3.08E-05 | 8 | 4 | TRUE | 0.3 | 500 | 0.01 | 2hrs, 5 mins | 0.9928 | 0.9676 | 1 | 1 |
| | 2 | GRID | 13/3/2025 | 12:39 PM | TRUE | 3.08E-05 | 8 | 4 | TRUE | 0.3 | 500 | 0.3 | 2hrs, 5 mins | 0.9921 | 0.9676 | 1 | 1 |
| | 3 | GRID | 13/3/2025 | 12:39 PM | TRUE | 3.08E-05 | 8 | 8 | TRUE | 0.3 | 500 | 0.01 | 2hrs, 5 mins | 0.9879 | 0.9459 | 0.9918 | 0.9919 |
| | 4 | GRID | 13/3/2025 | 12:39 PM | TRUE | 3.08E-05 | 8 | 8 | TRUE | 0.3 | 500 | 0.3 | 2hrs, 5 mins | 0.9879 | 0.9459 | 0.9918 | 0.9919 |
| | 5 | GRID | 13/3/2025 | 12:39 PM | TRUE | 3.08E-05 | 10 | 4 | TRUE | 0.3 | 500 | 0.01 | 2hrs, 5 mins | 0.9897 | 0.9595 | 0.9919 | 0.9919 |
| | 6 | GRID | 13/3/2025 | 12:39 PM | TRUE | 3.08E-05 | 10 | 4 | TRUE | 0.3 | 500 | 0.3 | 2hrs, 5 mins | 0.9897 | 0.9595 | 0.9918 | 0.9919 |
| | 7 | GRID | 13/3/2025 | 12:39 PM | TRUE | 3.08E-05 | 10 | 8 | TRUE | 0.3 | 500 | 0.01 | 2hrs, 5 mins | 0.9731 | 0.9324 | 0.9535 | 0.9539 |
| | 8 | GRID | 13/3/2025 | 12:39 PM | TRUE | 3.08E-05 | 10 | 8 | TRUE | 0.3 | 500 | 0.3 | 2hrs, 5 mins | 0.9731 | 0.9324 | 0.9535 | 0.9539 |
| | 9 | RANDOM | 13/3/2025 | 12:39 PM | TRUE | 3.08E-05 | 2 | 8 | TRUE | 0.3 | 800 | 0.1 | 2hrs, 5 mins | 0.9072 | 0.8548 | 0.9038 | 0.9 |
| | 10 | RANDOM | 13/3/2025 | 12:39 PM | TRUE | 1.8E-05 | 2 | 16 | FALSE | 0.3 | 1000 | 0.05 | 2hrs, 5 mins | 0.7833 | 0.7067 | 0.8561 | 0.8212 |
| | 11 | RANDOM | 13/3/2025 | 12:39 PM | FALSE | 9.08E-06 | 12 | 8 | FALSE | 0.1 | 1000 | 0.05 | 2hrs, 5 mins | 0.8404 | 0.7426 | 1 | 1 |
| | 12 | RANDOM | 13/3/2025 | 12:39 PM | FALSE | 3.08E-05 | 2 | 12 | TRUE | 0.1 | 500 | 0.3 | 2hrs, 5 mins | 0.8081 | 0.6387 | 0.9538 | 1 |
| | 13 | RANDOM | 13/3/2025 | 12:39 PM | TRUE | 1.8E-05 | 10 | 32 | TRUE | 0.1 | 100 | 0.01 | 2hrs, 5 mins | 0.842 | 0.6739 | 1 | 1 |
| | 14 | RANDOM | 13/3/2025 | 12:39 PM | TRUE | 5.08E-05 | 12 | 24 | TRUE | 0.2 | 200 | 0 | 2hrs, 5 mins | 0.919 | 0.921 | 1 | 1 |
| | 15 | RANDOM | 13/3/2025 | 12:39 PM | FALSE | 1.08E-05 | 10 | 4 | TRUE | 0.3 | 1000 | 0.01 | 2hrs, 5 mins | 0.9921 | 0.9676 | 1 | 1 |
| | 16 | RANDOM | 13/3/2025 | 12:39 PM | FALSE | 3.08E-05 | 10 | 16 | FALSE | 0.3 | 1000 | 0.4 | 2hrs, 5 mins | 0.9503 | 0.907 | 0.9921 | 0.9921 |
| | 17 | RANDOM | 13/3/2025 | 12:39 PM | TRUE | 3.08E-05 | 2 | 12 | TRUE | 0.2 | 100 | 0.1 | 2hrs, 5 mins | 0.9203 | 0.8177 | 0.9597 | 0.9597 |
| | 18 | RANDOM | 13/3/2025 | 12:39 PM | FALSE | 1.8E-05 | 4 | 24 | TRUE | 0.2 | 200 | 0.05 | 2hrs, 5 mins | 0.9597 | 0.8992 | 0.9908 | 0.9908 |

Fig. 26. Experimentation Logs of 3 Assigned Hyperparameters

The thorough examination of 18 experimental logs makes it clear that num_train_epochs, test_size, and weight_decay are the main factors that influence the quality of the embedding and the retrieval performance. In all cases, training for only two epochs led to underfitting, as demonstrated by MRR values below 0.92 and Top-1 accuracy under 0.88, thus indicating that the model did not learn semantic relationships adequately. By training for eight, ten, or twelve epochs, the results improved drastically, as confirmed by MRR values and Top-1 accuracy exceeding 0.95 and 0.92 respectively, thus highlighting the importance of training long enough to achieve stable embeddings.

The percentage of data set aside for testing was also a factor that strongly influenced the results. Test sizes between 0.2 and 0.3 yielded the best retrieval metrics whereas smaller splits limited generalization and larger splits slightly reduced MRR thus indicating the necessity of balancing training and validation data. The weight decay of 0.01 was the best regularization, thus enhancing embedding stability and retrieval accuracy. On the other hand, extreme values either over- or under-regularized the model, thus causing the occasional performance drops.

In addition, the hyperparameters such as learning rate, batch size, mixed-precision usage, shuffle, and warmup steps affected the training speed, convergence, and runtime, however, their influence on the final retrieval metrics was very minimal. The main takeaways from this study are that num_train_epochs, test_size, and weight_decay are the most critical factors to consider when aiming at high-quality embeddings and that Grid Search can be used to find the optimal configurations while Random Search can be used to verify the stability of the model for various combinations of parameters.

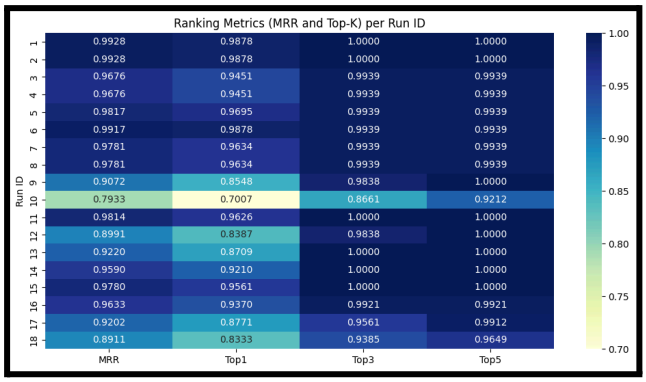


Fig. 27. MRR and Top K per Run ID

The heatmap outlines the 18 experimental runs with ranking metrics MRR, Top-1, Top-3, and Top-5 for each run. The lighter blue shades indicate higher performance, thus, the top configurations can be quickly identified. In general, GRID Search runs have led to higher performance, especially those with num_train_epochs of 10, weight_decay of 0.01 or 0.3, and test_size of 0.3, where the highest MRR was 0.9928 and Top-K metrics were close to 1.

RANDOM Search had more variables, with some configurations achieving MRR up to 0.9780 and Top-1 accuracy of 0.9561, whereas those with fewer epochs and weight decay in the wrong direction had MRR as low as 0.7933. This heatmap serves to prove that longer training times, balanced test sizes, and correct regularization not only improve the quality of embeddings but also increase the performance of retrieval are true for both search strategies.

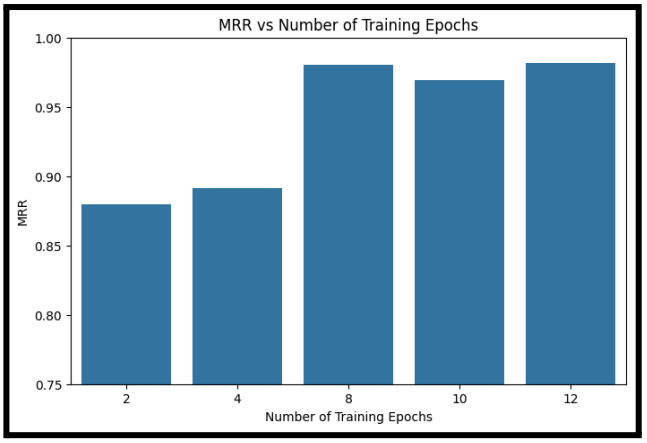


Fig. 28. MRR value per num_train_epochs

The bar chart illustrates the connection between num_train_epochs and MRR. The rise is very evident, which suggests that the model becomes better at capturing semantic relations in the embedding space as the number of training epochs is increased. The MRR values corresponding to only two epochs were very low, such as 0.7933 and 0.9072, thus, these runs showed signs of underfitting. The models with ten training epochs reached the highest MRR values, even up to 0.9780. The present point is to a large extent made through the figure: it is very important to choose the right number of training epochs in order to get the best retrieval performance and at the same time not overfit.

## V. Conclusion

The study reconfigured a MiniLM model to perform semantic retrieval on a book dataset by experiment hyperparameter tuning in a systematic manner. The idea was to find the best combination of num_train_epochs, weight_decay, test_size, learning_rate, per_device_train_batch_size, shuffle, warmup_steps, and mixed-precision (fp16) attributes that maximize the model's performance. Out of 18 experimental runs it was found that the highest retrieval performance was obtained for a grid search configuration with the parameters: num_train_epochs=10, weight_decay=0.01, test_size=0.3, learning_rate=3.00E-05, batch size=4, shuffle=True, 500 warmup steps and fp16 enabled. The performance scores in this configuration are MRR=0.9928, Top-1 accuracy=0.9878, and Top-3/Top-5 accuracies=1.0000, which shows almost perfect retrieval of semantically related book titles.

Moreover, the different hyperparameters that were tested under this study such as learning rate, epochs, and weight_decay had a significant effect on the quality of the embedding. Increasing training epochs led to continuous MRR and Top-K metric improvements. At the same time extremely high or low weight_decay values had an adverse effect on model performance. The best test sizes of between 0.2 and 0.3 were a good compromise between training and evaluation as they ensured enough

data for the model to generalize. Variations of the learning rate had an impact on convergence stability and batch sizes for each device affected runtime and precision when combined with fp16 mixed-precision training. The shuffle parameter and warmup steps were facilitators of training stability while different batches with larger sizes or an extreme weight decay might result in less effective performances even though the training was longer.

Better search strategies can explain the differences in results between runs and the comparison done in the study is the proof of this. Grid Search was able to provide high and stable retrieval metrics for most of the executed runs while Random Search was more inherently variable and peak performance was generally lower. Random Search is a good choice for fast exploration of wide hyperparameter spaces but Grid Search's structured approach was more efficient in pinpointing the optimal hyperparameter configuration for the given data set.

Certain limitations and issues have been identified alongside successful optimization. Long runtimes were a major impediment for hyperparameter experiments since they often took hours per configuration and the total number of possible experiments were limited due to the availability of time.

The study demonstrates that precise adjustment of all necessary hyperparameters that involve epochs, weight_decay, test_size, learning_rate, batch size, shuffle, warmup steps, and fp16 can dramatically increase the quality of embeddings and the effectiveness of semantic retrieval. There is a lot of scope for further research to broaden the search space, use multi-task fine-tuning, or even leverage larger datasets for better model generalization and robustness. If one wants maximum performance and reproducibility, then Grid Search is advisable, whereas Random Search can still be used for quick and brief tuning sessions.

## VI. References


[1] C. D. Manning, P. Raghavan, and H. Schütze, Introduction to Information Retrieval. Cambridge University Press, 2008. [Online]. Available: https://nlp.stanford.edu/IR-book/pdf/irbookonlinereading.pdf
[Accessed: 08-Nov-2025].

[2] Sentence-Transformers, "Training Dense Retrievers with Multiple Negatives Ranking Loss (MNRL)," 2024. [Online]. Available: https://sbert.net/examples/sparse_encoder/training/retrievers/README.html
[Accessed: 08-Nov-2025].

[3] D. Castillo, "7k Books – with Metadata," Kaggle Data, 2024. [Online]. Available: https://www.kaggle.com/datasets/dylanjcastillo/7k-books-with-metadata
[Accessed: 08-Nov-2025].

[4] Will Koehrsen, "Intro to Model Tuning: Grid and Random Search," Kaggle, 2025. [Online]. Available: https://www.kaggle.com/code/willkoehrsen/intro-to-model-tuning-grid-and-random-search
[Accessed: 08-Nov-2025].

[5] G. Izacard et al., "Unsupervised Dense Information Retrieval with Contrastive Learning," arXiv preprint arXiv:2112.09118, 2021. [Online]. Available: https://arxiv.org/abs/2112.09118
[Accessed: 08-Nov-2025].

[6] Towards Data Science, "SBERT – Parameter-Efficient Transformer-Based Deep Sentence Embeddings," 2023. [Online]. Available: https://towardsdatascience.com/sbert-deb3d4aef8a4
[Accessed: 08-Nov-2025].

[7] N. Selby, "How many epochs is enough — Factors influencing epoch count," Nebius Blog, 2 Jul 2025. [Online]. Available: https://nebius.com/blog/posts/factors-influencing-epoch-count
[Accessed: 08-Nov-2025].